OOP Course Quiz

Assessment covering class diagrams, design patterns, sequence diagrams, and use case diagrams.

1. Review the diagram below.

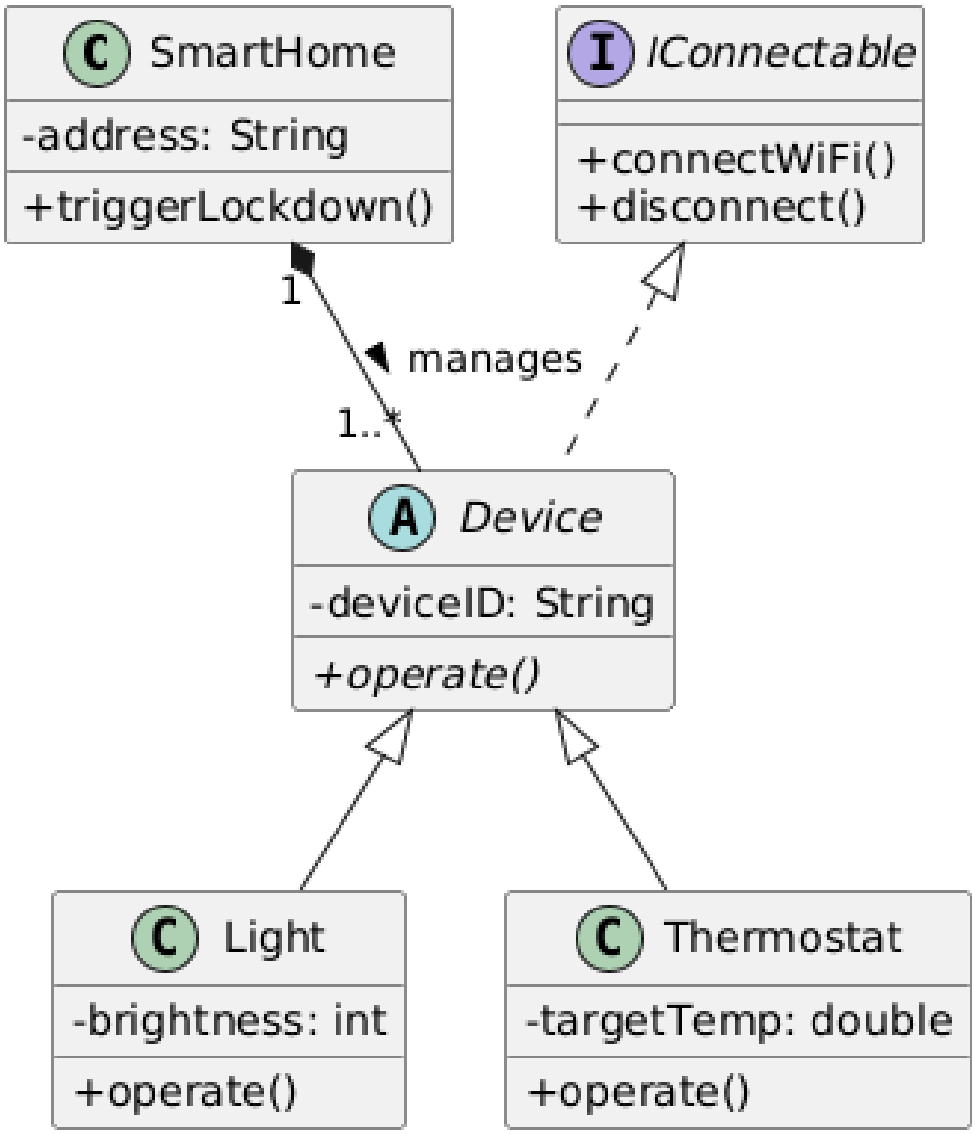

Diagram 1

What type of relationship is represented by the solid diamond between SmartHome and Device?

a) Aggregation
b) **Composition**
c) Generalization
d) Association

2. Review the diagram below.

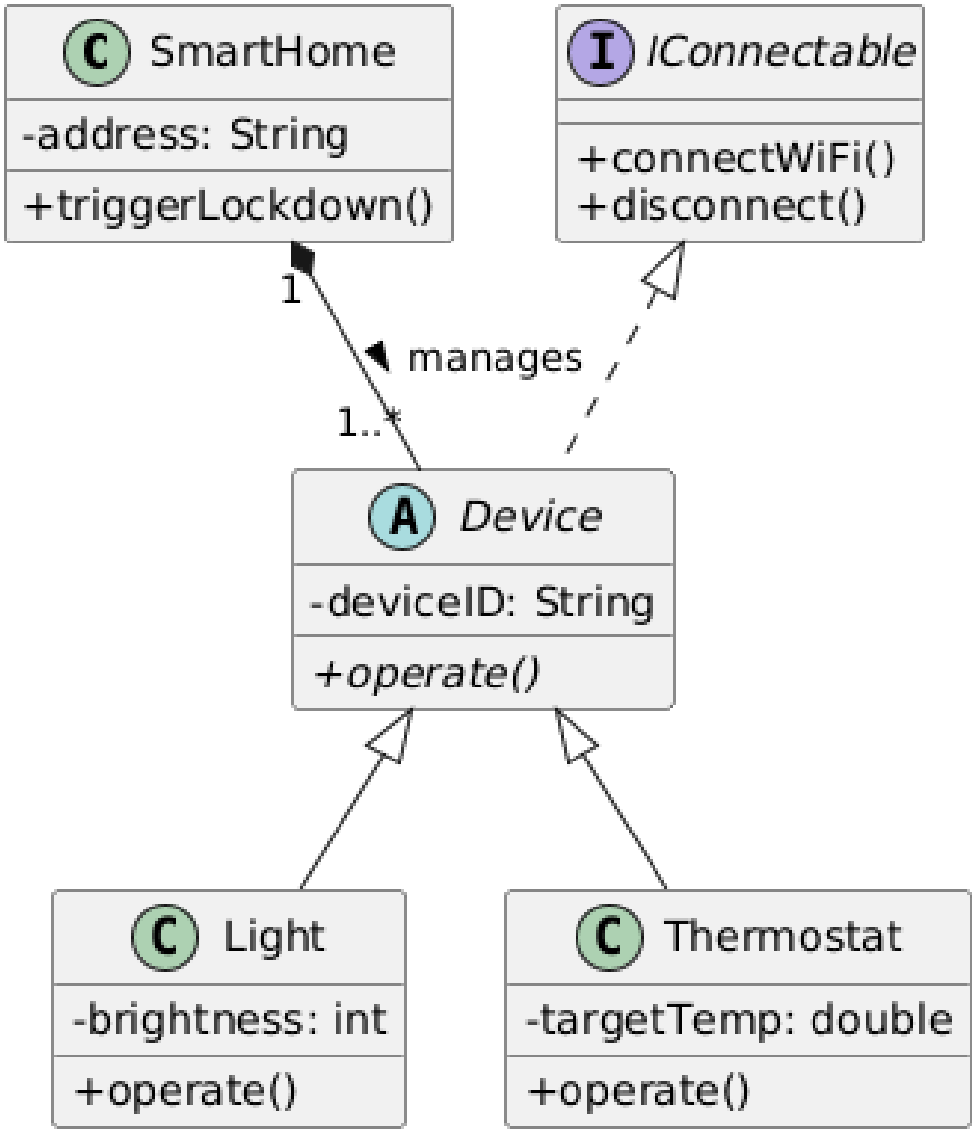

Diagram 1

Why is the `Device` class marked as abstract?

a) It contains only private methods.

b) **It is a base class that cannot be directly instantiated, requiring subclasses to define specific behaviors.**

c) It prevents the class from having any attributes.

d) It allows the class to inherit from multiple interfaces.

3. Review the diagram below.

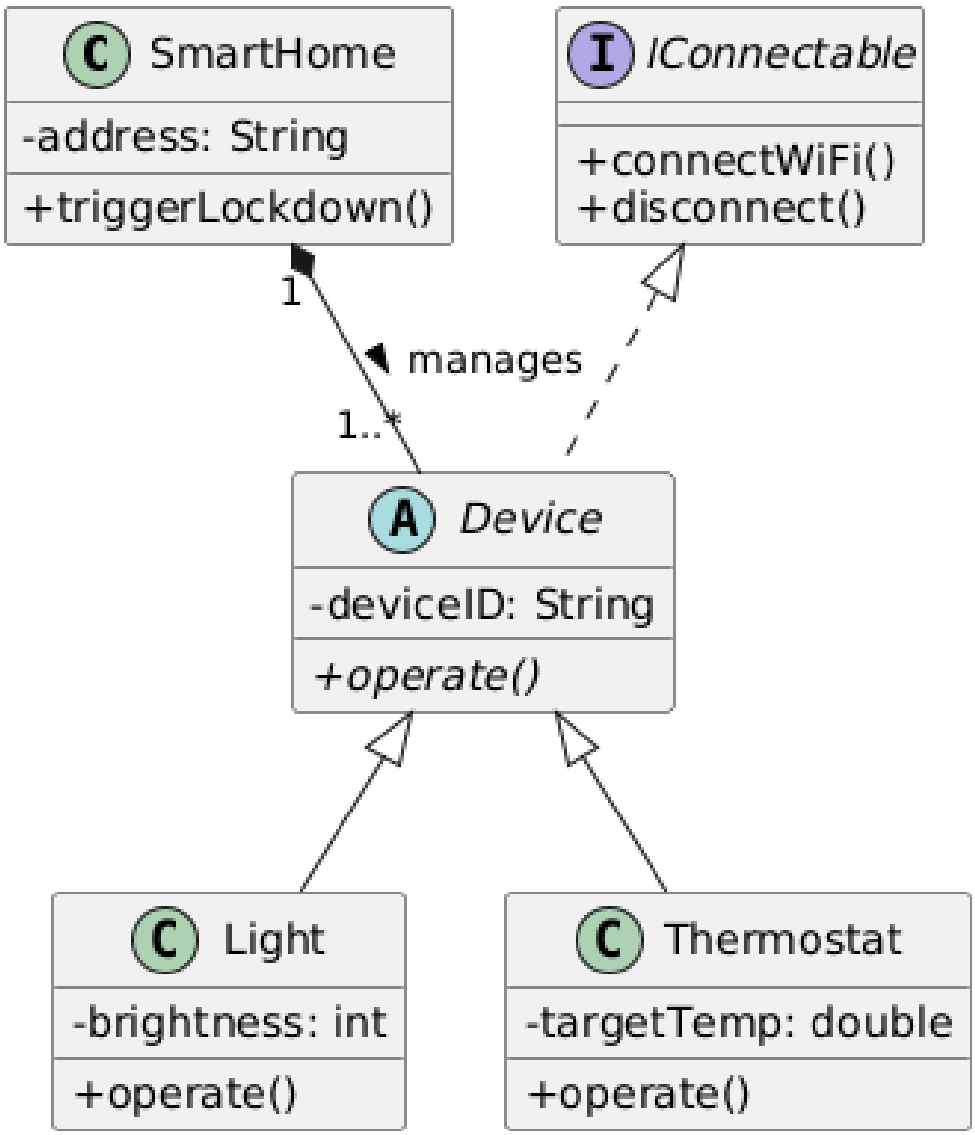

Diagram 1

What does the multiplicity "1..*" next to the Device class indicate?

a) A SmartHome can have exactly one Device.

b) **A SmartHome must contain at least one Device, but can contain many.**

c) A Device can belong to multiple SmartHomes.

d) A SmartHome can exist with zero Devices.

4. Review the diagram below.

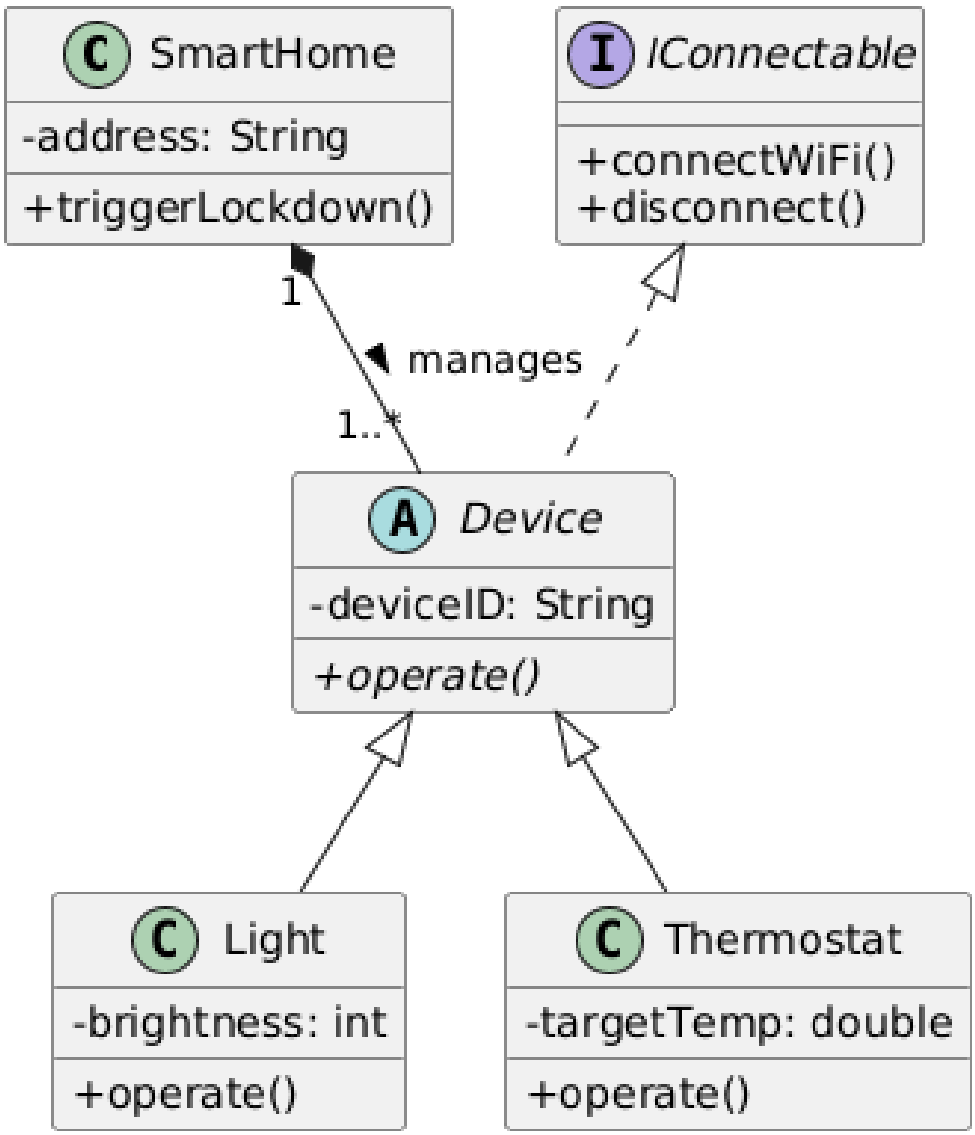

Diagram 1

Because the `Device` class implements the `IConnectable` interface, what is required of the `Light` class?

a) It must create its own separate interface.

b) It can choose whether or not to include networking methods.

c) **It inherits the interface contract and must have implementations for the `connectWiFi()` and `disconnect()` methods.**

d) It must override the `SmartHome` attributes.

5. Review the diagram below.

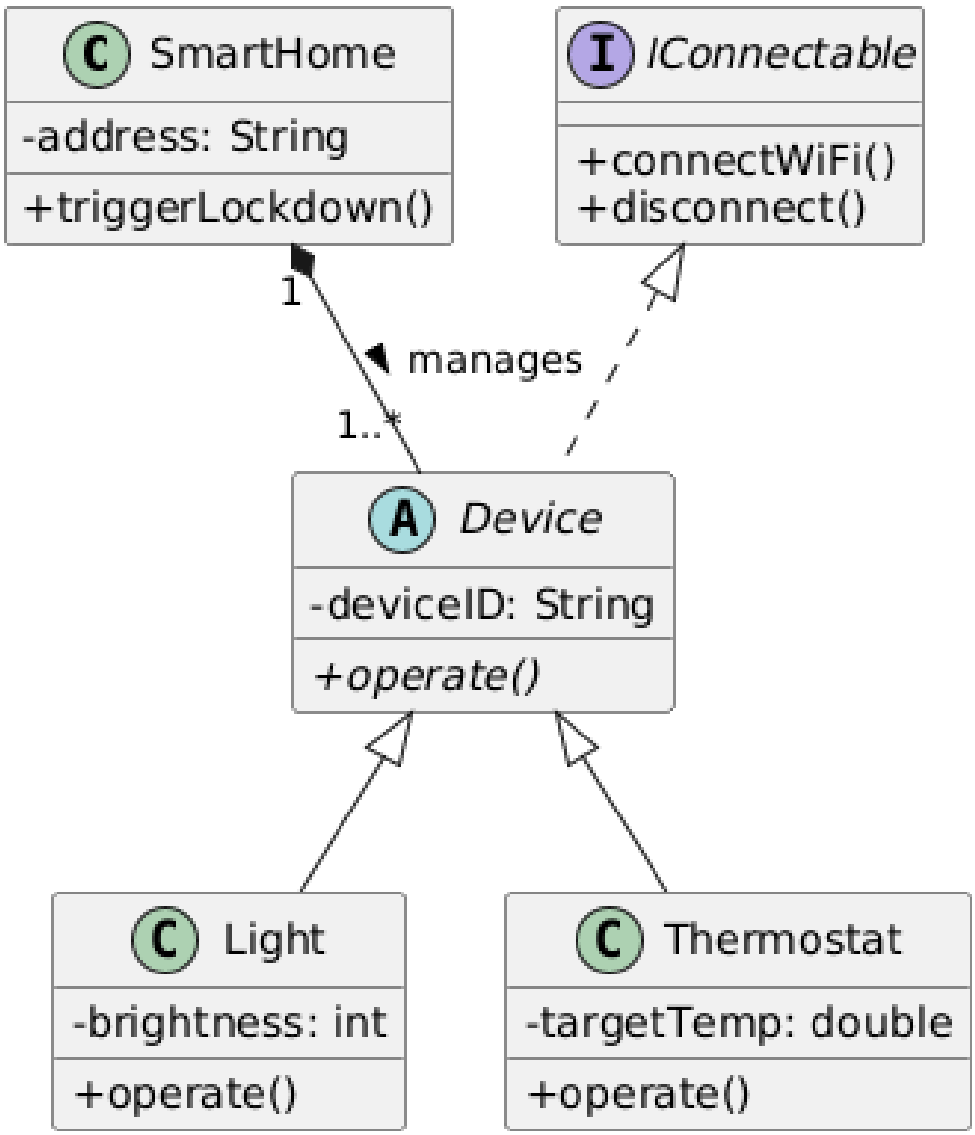

Diagram 1

If a program calls the `operate()` method on a `Device` variable that currently holds a `Thermostat` object, which concept ensures the `Thermostat`'s specific version of the method runs?

a) **Polymorphism**
b) Encapsulation
c) Method Overloading
d) Multiple Inheritance

6. Review the diagram below.

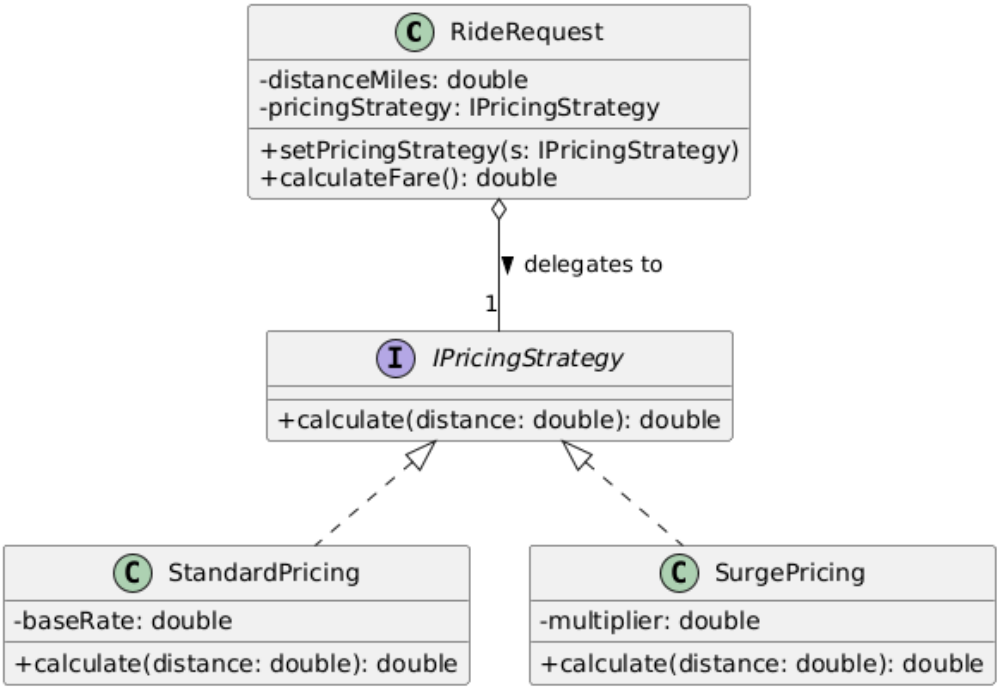

Diagram 2

What is the primary benefit of delegating the fare calculation to the `IPricingStrategy` interface?

a) It makes the code run faster.

b) **It allows new pricing algorithms to be added without modifying the `RideRequest` class.**

c) It allows `RideRequest` to inherit from multiple classes.

d) It hides the distance variable from the user.

7. Review the diagram below.

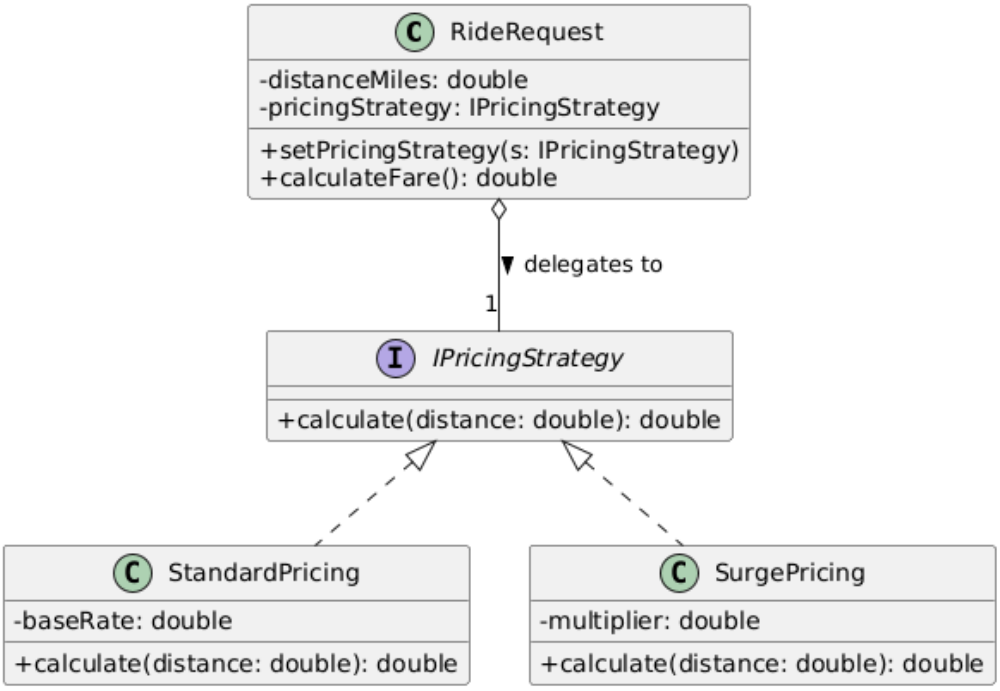

Diagram 2

In this diagram, what exactly is `IPricingStrategy`?

a) A concrete class that contains the math for calculating fares.

b) **An interface that acts as a blueprint, defining a method signature that other classes must implement.**

c) A subclass that inherits from `RideRequest`.

d) An external database that stores standard pricing rates.

8. Review the diagram below.

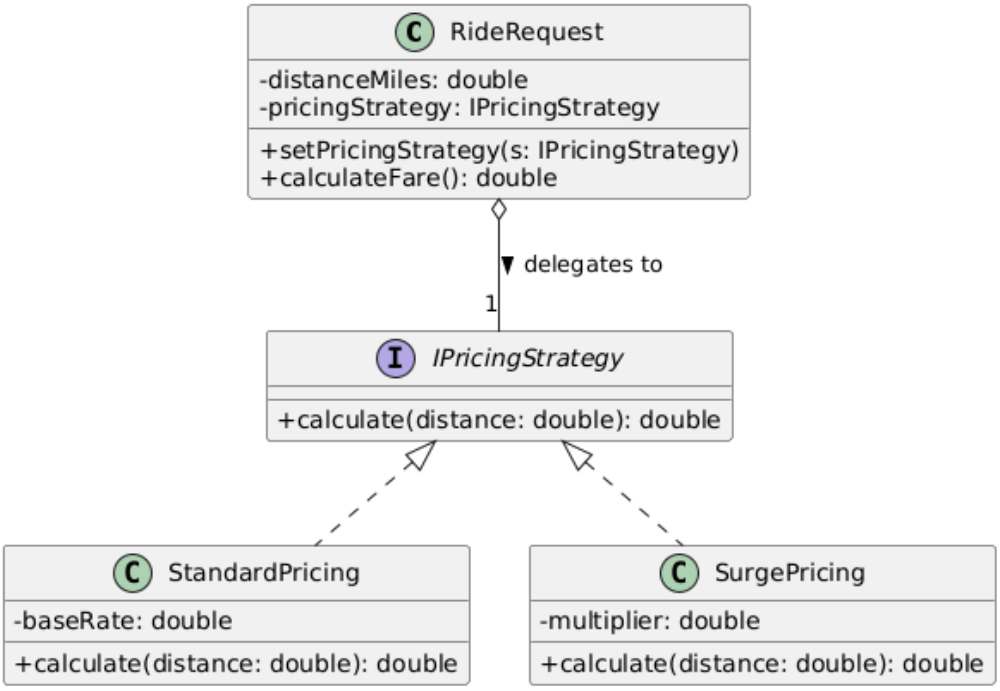

Diagram 2

What does the empty diamond arrow connecting `RideRequest` to `IPricingStrategy` represent?

a) Inheritance

b) **Aggregation**

c) Composition

d) Realization

9. Review the diagram below.

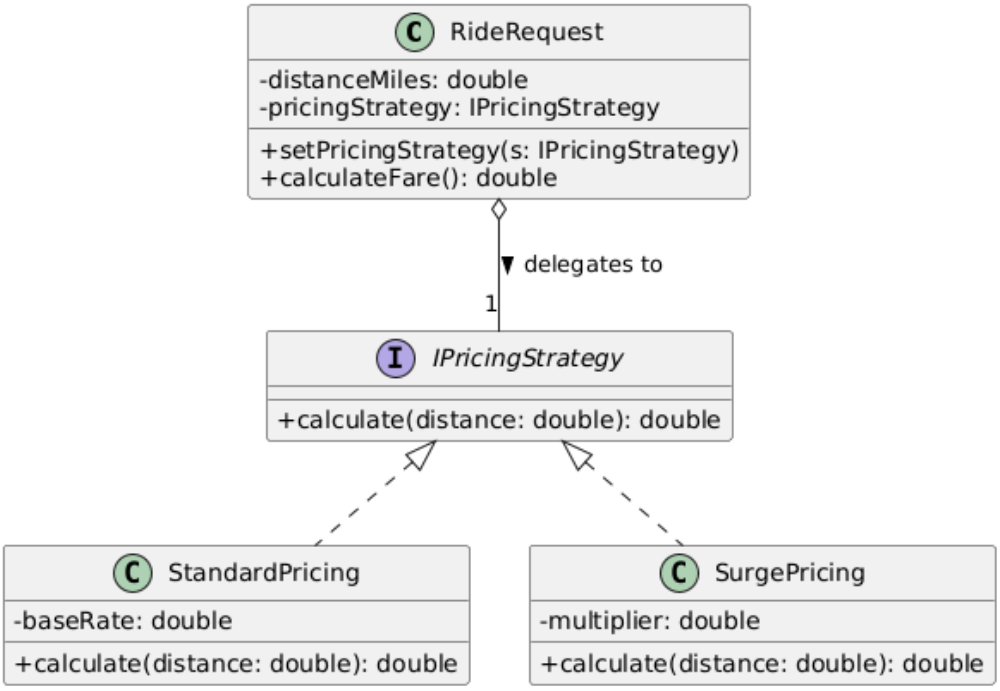

Diagram 2

When the `RideRequest` class executes the `calculate()` method, how does the program know whether to use the math from `StandardPricing` or `SurgePricing`?

a) The `RideRequest` class contains an internal if/else statement to decide.

b) It runs both methods and adds the results together.

c) **It executes the method belonging to the specific object type (Standard or Surge) that is currently assigned to the `pricingStrategy` variable.**

d) The interface calculates it automatically.

10. Review the diagram below.

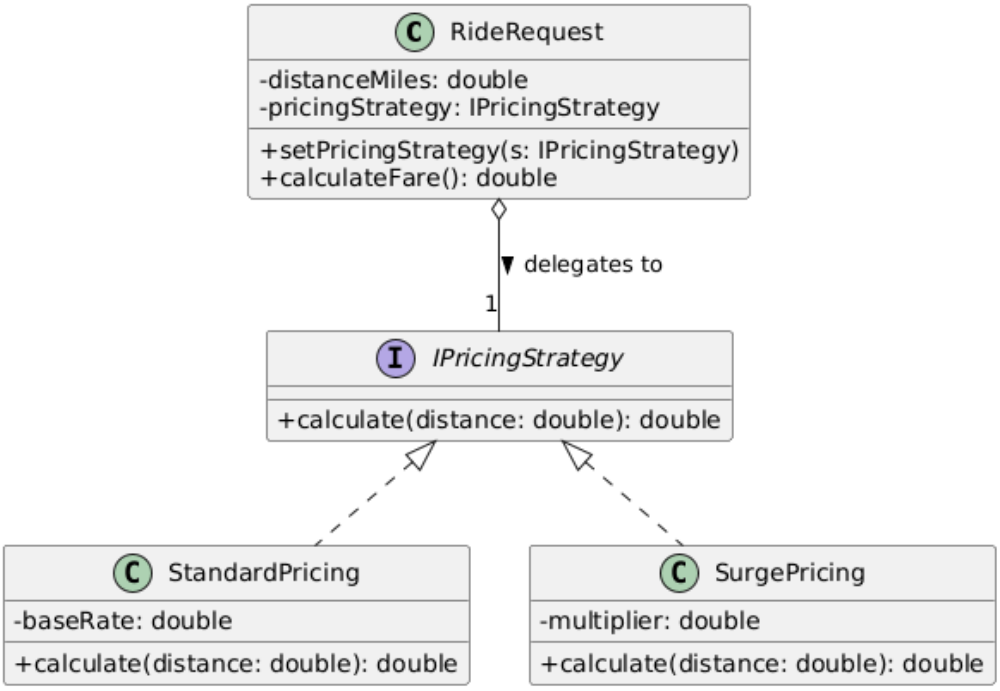

Diagram 2

How would you add a `HolidayPricing` calculation to this system?

a) Add a new `if/else` statement inside `RideRequest`.

b) Modify the `StandardPricing` class to include holiday logic.

c) Change `IPricingStrategy` into an abstract class.

d) **Create a new `HolidayPricing` class that implements `IPricingStrategy`.**

11. Review the diagram below.

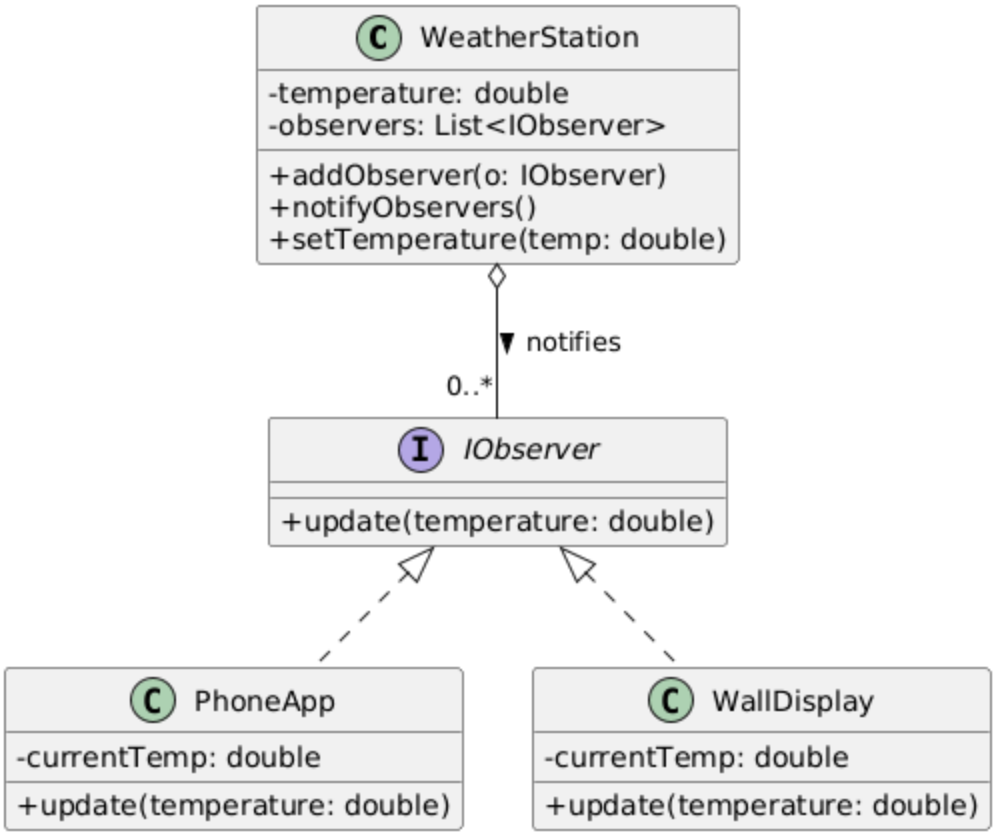

Diagram 3

According to the diagram, what type of data is stored in the `observers` attribute of the `WeatherStation` class?

a) A list of temperature values.

b) A single `PhoneApp` object.

c) **A list of objects, as long as those objects implement the `IObserver` interface.**

d) A list of strings containing weather forecasts.

12. Review the diagram below.

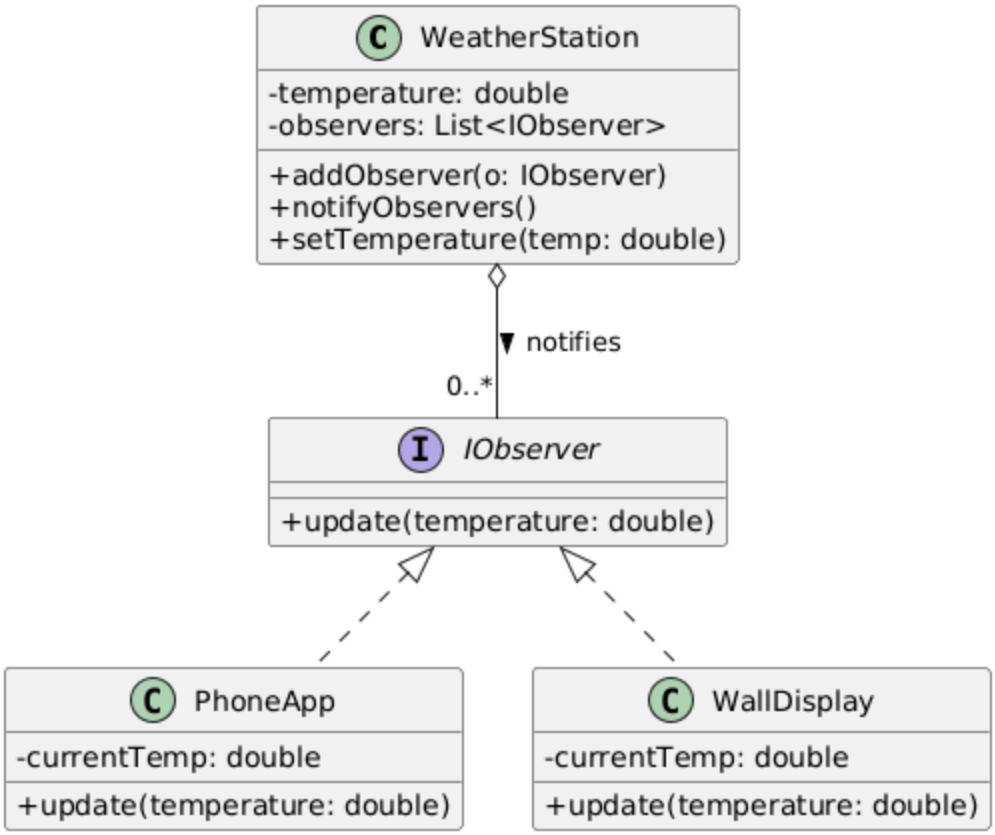

Diagram 3

Both `PhoneApp` and `WallDisplay` contain an `update(temperature: double)` method. Why is this required by the architecture?

a) They inherit the method directly from the `WeatherStation` class.

b) It is a coincidence that they share the same method name.

c) **They both implement the `IObserver` interface, which forces them to define their own version of this method.**

d) `PhoneApp` is a parent class to `WallDisplay`.

13. Review the diagram below.

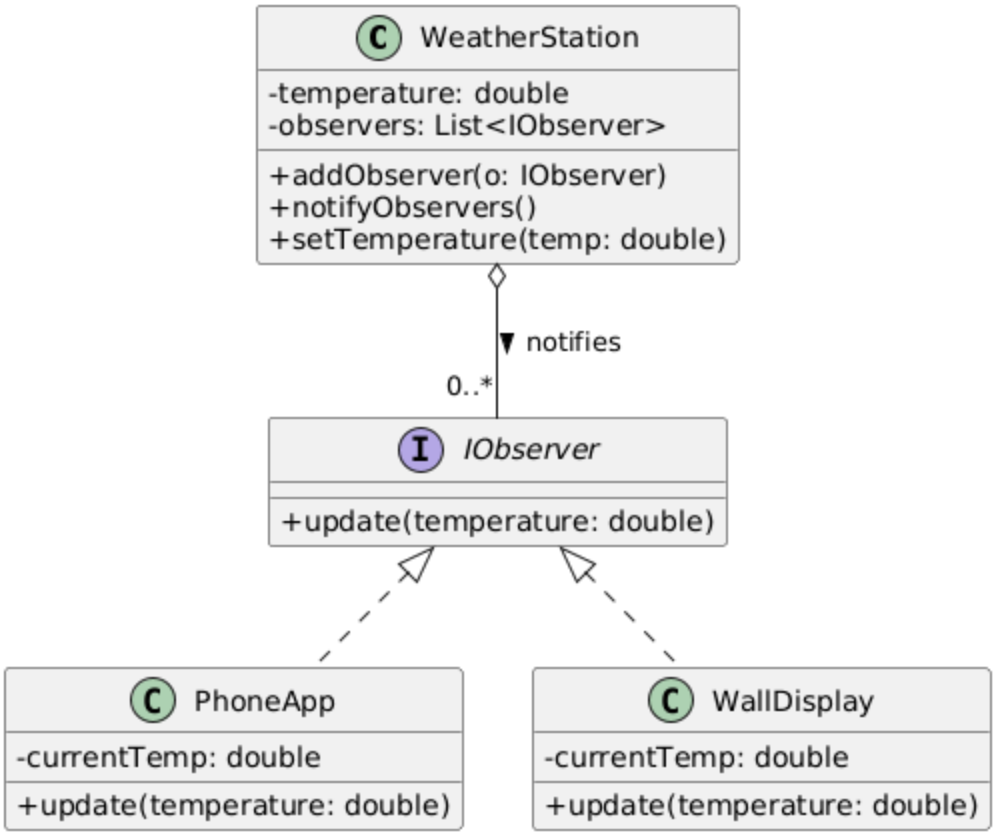

Diagram 3

Why does the `WeatherStation` maintain a list of `IObserver` interfaces rather than concrete classes like `PhoneApp`?

a) Because interfaces use less memory.

b) **To reduce coupling, allowing the station to notify any object that implements the interface without knowing its specific type.**

c) Because Java and C++ require lists to hold interfaces.

d) To prevent the observers from modifying the temperature.

14. Review the diagram below.

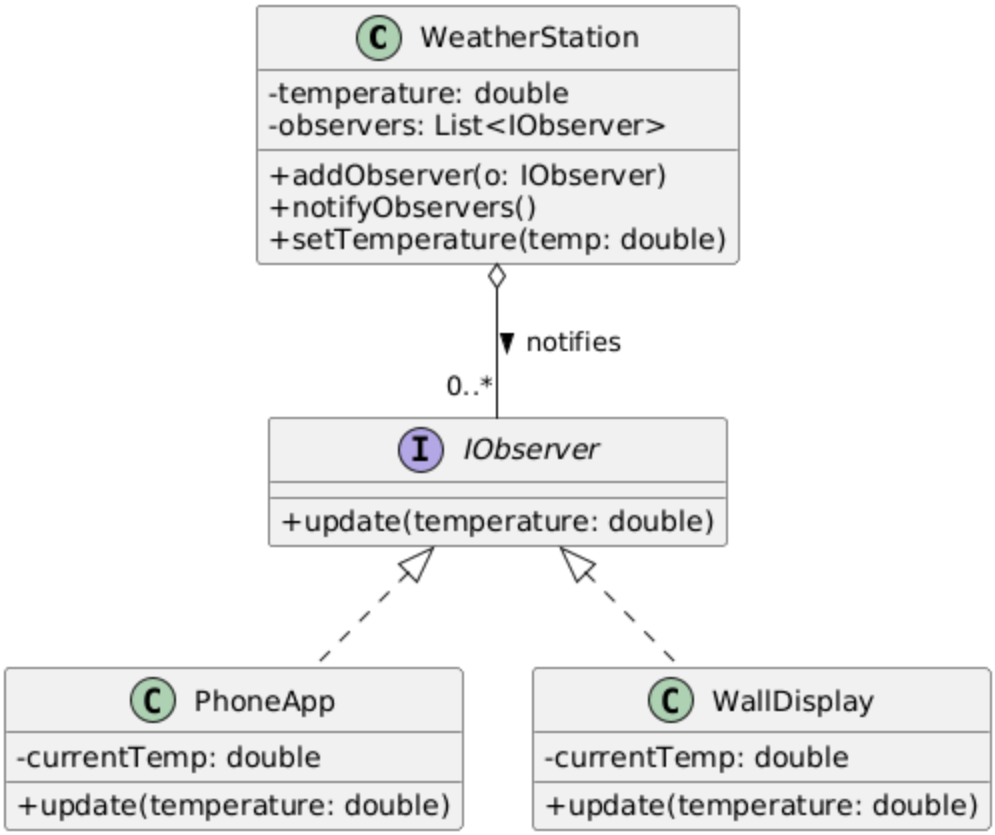

Diagram 3

What does the dashed line with the unfilled arrow pointing from `PhoneApp` to `IObserver` mean?

a) **The class implements (realizes) the interface.**

b) The class inherits from another class.

c) The class contains the interface.

d) The class depends on the interface temporarily.

15. Review the diagram below.

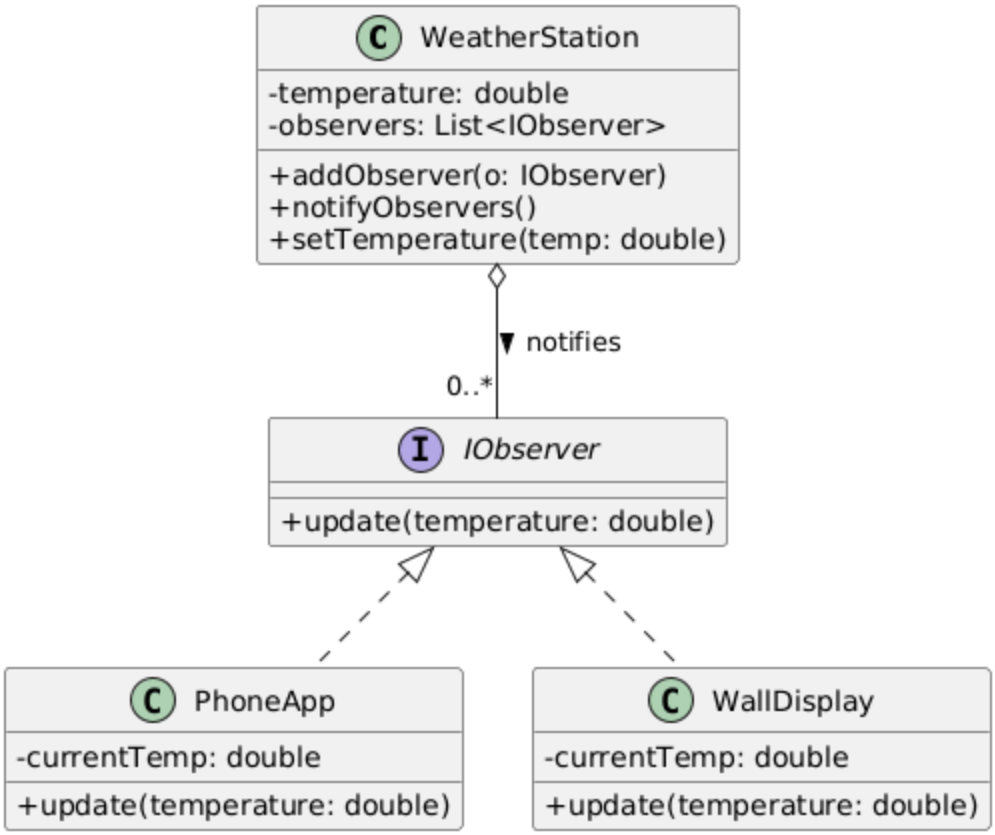

Diagram 3

In the context of this pattern, the `WeatherStation` is typically referred to as the:

a) Listener
b) Subscriber
c) **Subject**
d) Strategy

16. Review the diagram below.

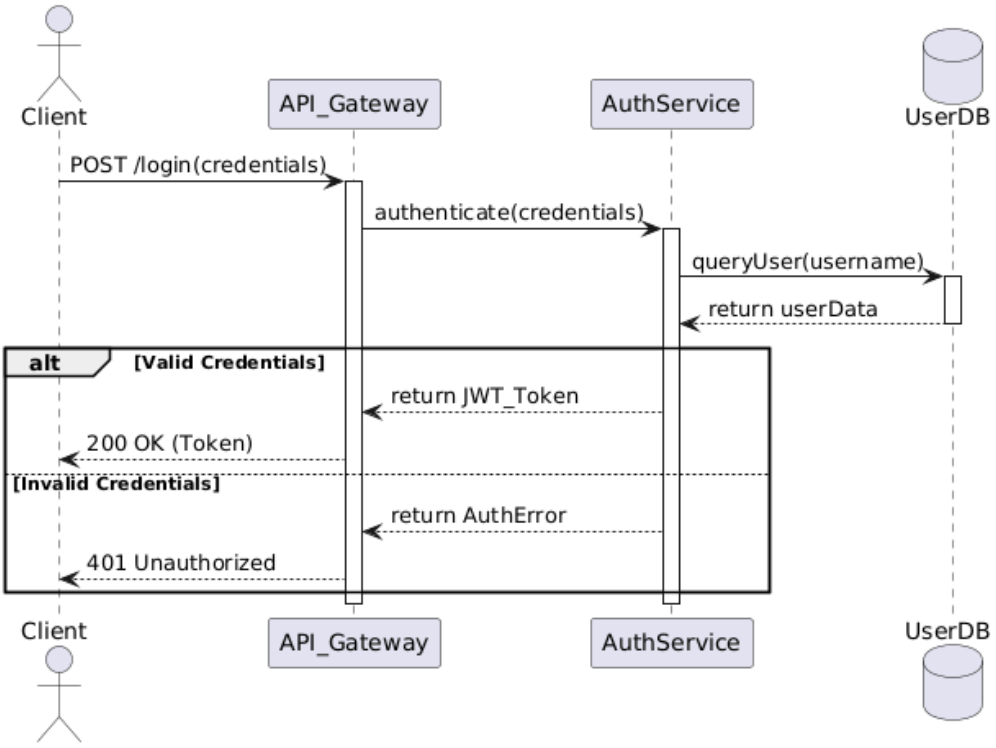

Diagram 4

In a sequence diagram, what does a solid line with a solid arrowhead represent?

a) **A synchronous message or method call**
b) An asynchronous message
c) A return message
d) An object creation

17. Review the diagram below.

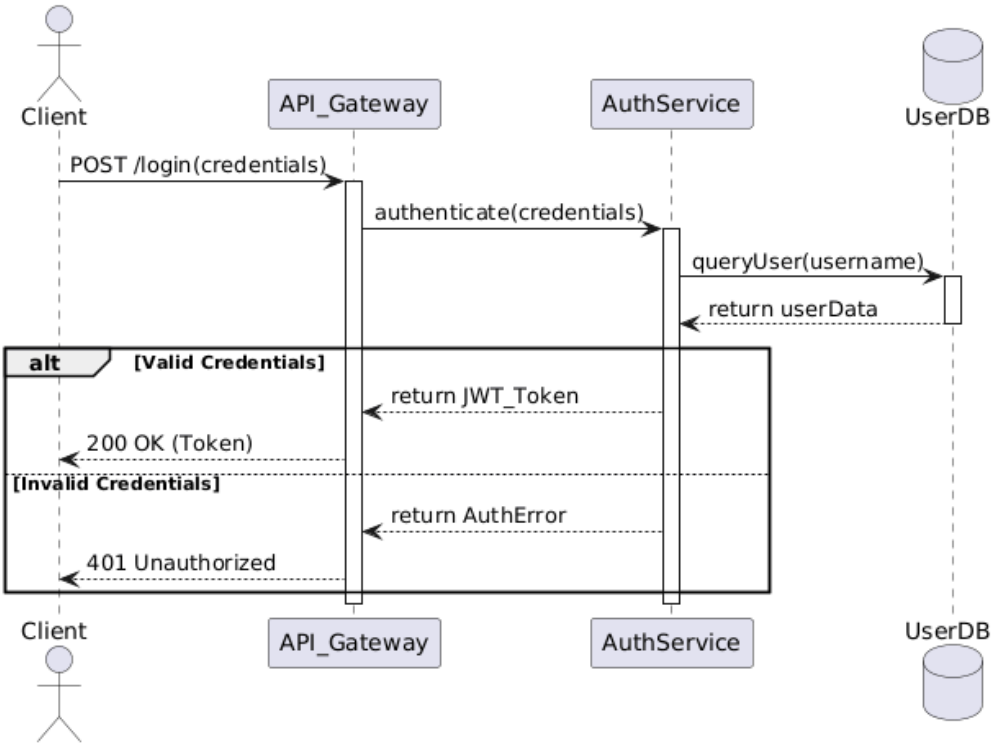

Diagram 4

What do the vertical rectangular boxes on the lifelines indicate?

a) The object is being created.

b) **The object is active and currently processing a method or task.**

c) The object has been destroyed.

d) The object is waiting for user input.

18. Review the diagram below.

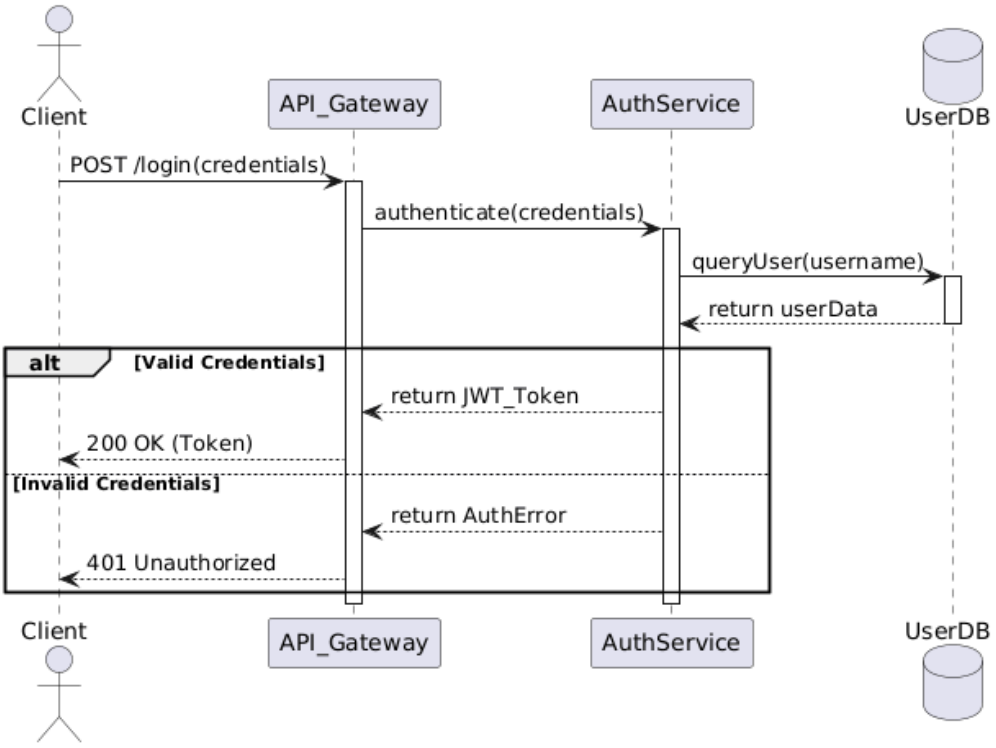

Diagram 4

Reading the sequence diagram from top to bottom, what action happens immediately after the `AuthService` sends the `queryUser(username)` message to the `UserDB`?

a) The `Client` logs in.

b) The `API_Gateway` authenticates the user.

c) **The `UserDB` returns `userData` back to the `AuthService`.**

d) The `AuthService` returns a `JWT_Token` to the `API_Gateway`.

19. Review the diagram below.

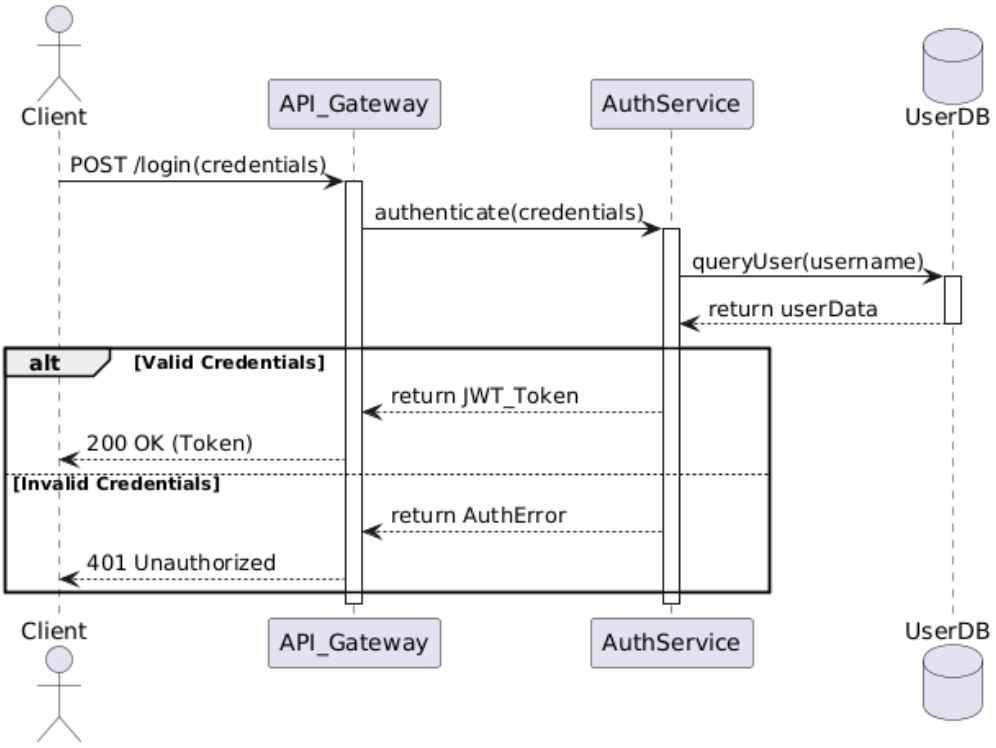

Diagram 4

According to the diagram, which component is responsible for querying the `UserDB`?

a) The Client
b) The API_Gateway
c) **The AuthService**
d) The database queries itself

20. Review the diagram below.

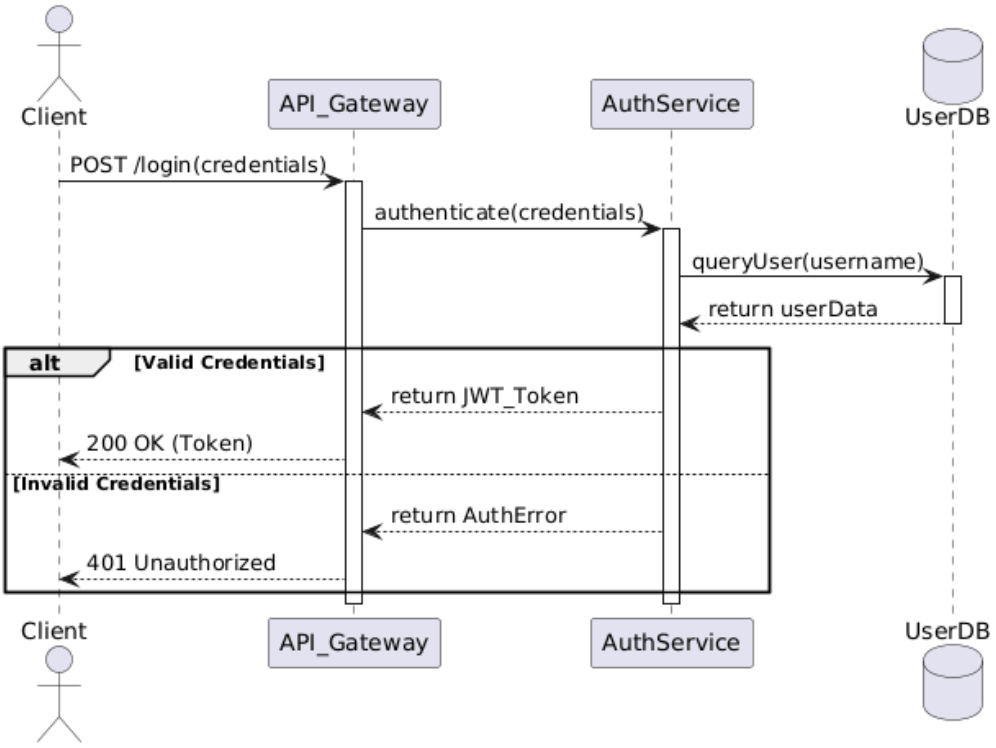

Diagram 4

What type of message is represented by the dashed arrows returning to the left?

a) A new method call

b) **A return message with a result**

c) An asynchronous broadcast

d) A database update

21. Review the diagram below.

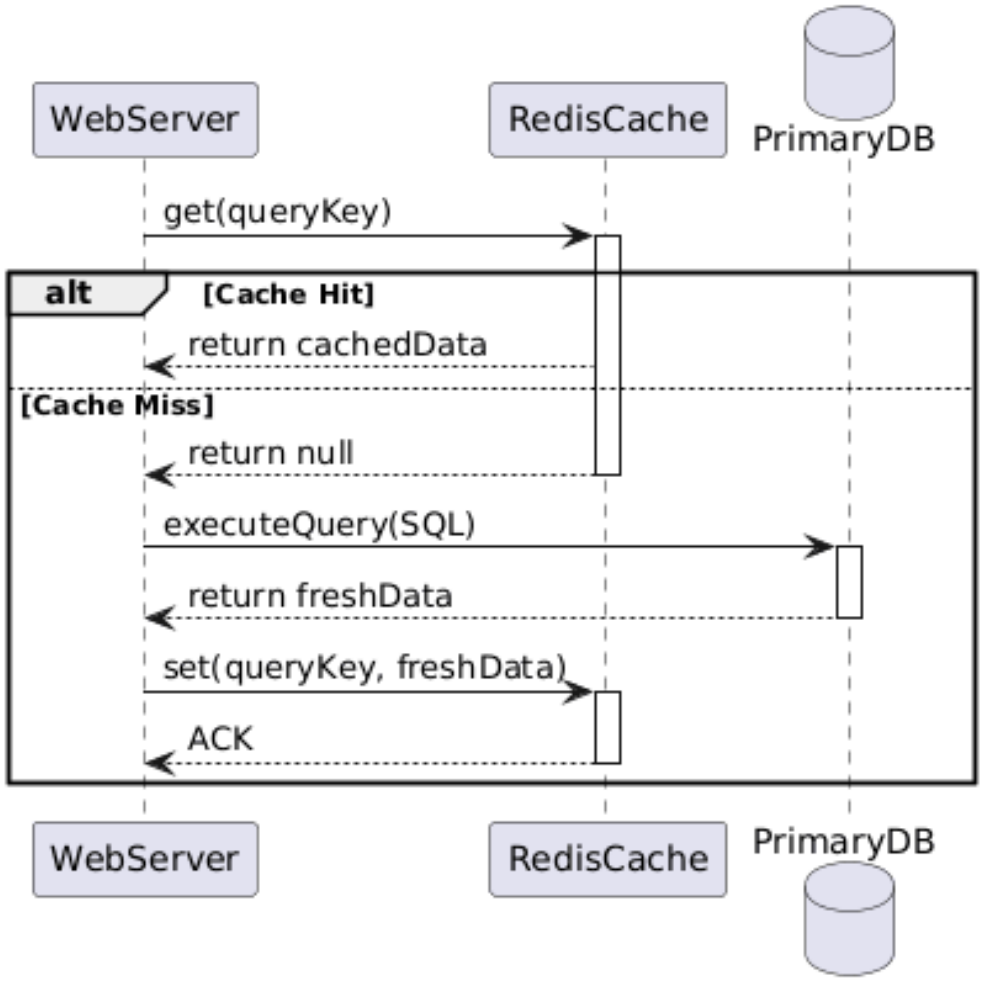

Diagram 5

Under what specific condition does the sequence diagram show a message being sent to the `PrimaryDB`?

a) When the WebServer first starts.

b) Every time a request is made.

c) **Only when a Cache Miss occurs (the data is not in Redis).**

d) Only when a Cache Hit occurs.

22. Review the diagram below.

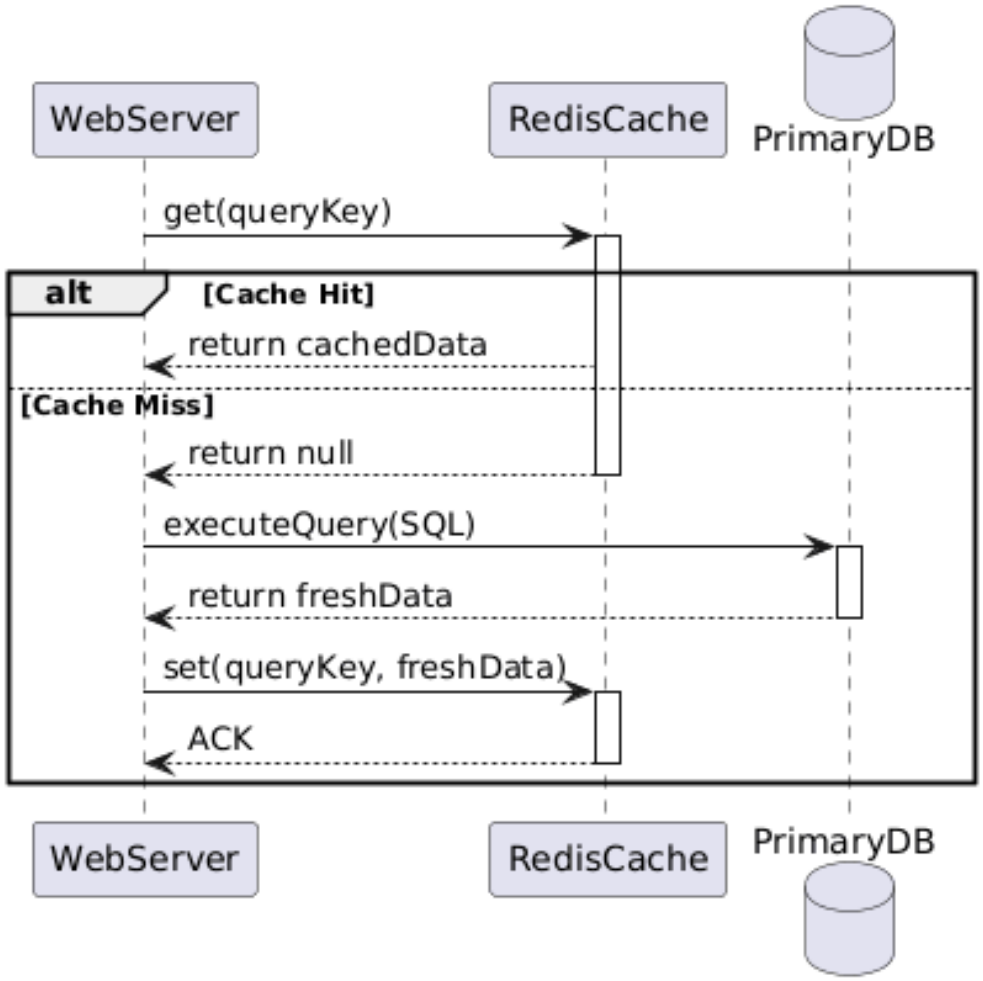

Diagram 5

What happens immediately after the `PrimaryDB` returns `freshData` to the `WebServer`?

a) The `WebServer` sends the data to the client.

b) The `RedisCache` is cleared.

c) **The `WebServer` saves the `freshData` into the `RedisCache` using a `set` command.**

d) The sequence ends.

23. Review the diagram below.

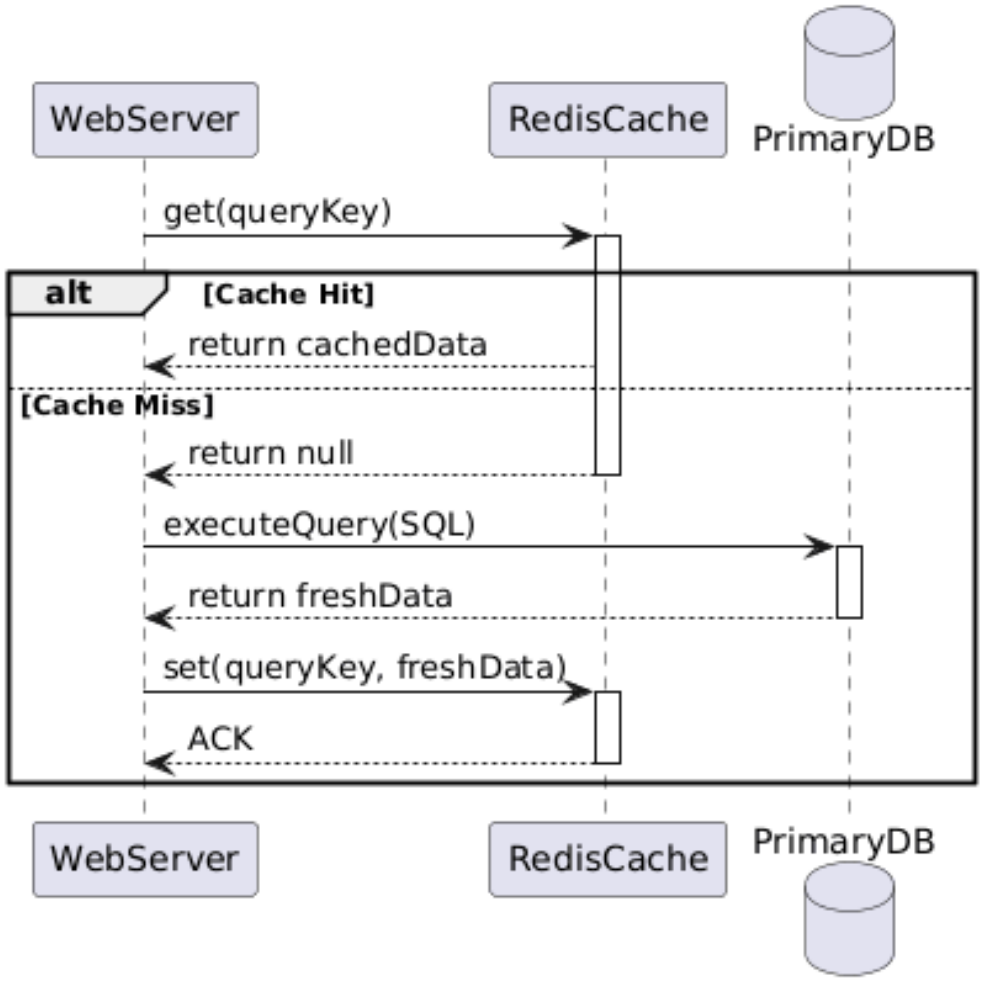

Diagram 5

What is the primary purpose of a sequence diagram?

a) **To visualize the order of messages passed between objects over time.**
b) To show the static relationships and attributes of classes.
c) To display the physical deployment of hardware.
d) To illustrate the state changes of a single object.

24. Review the diagram below.

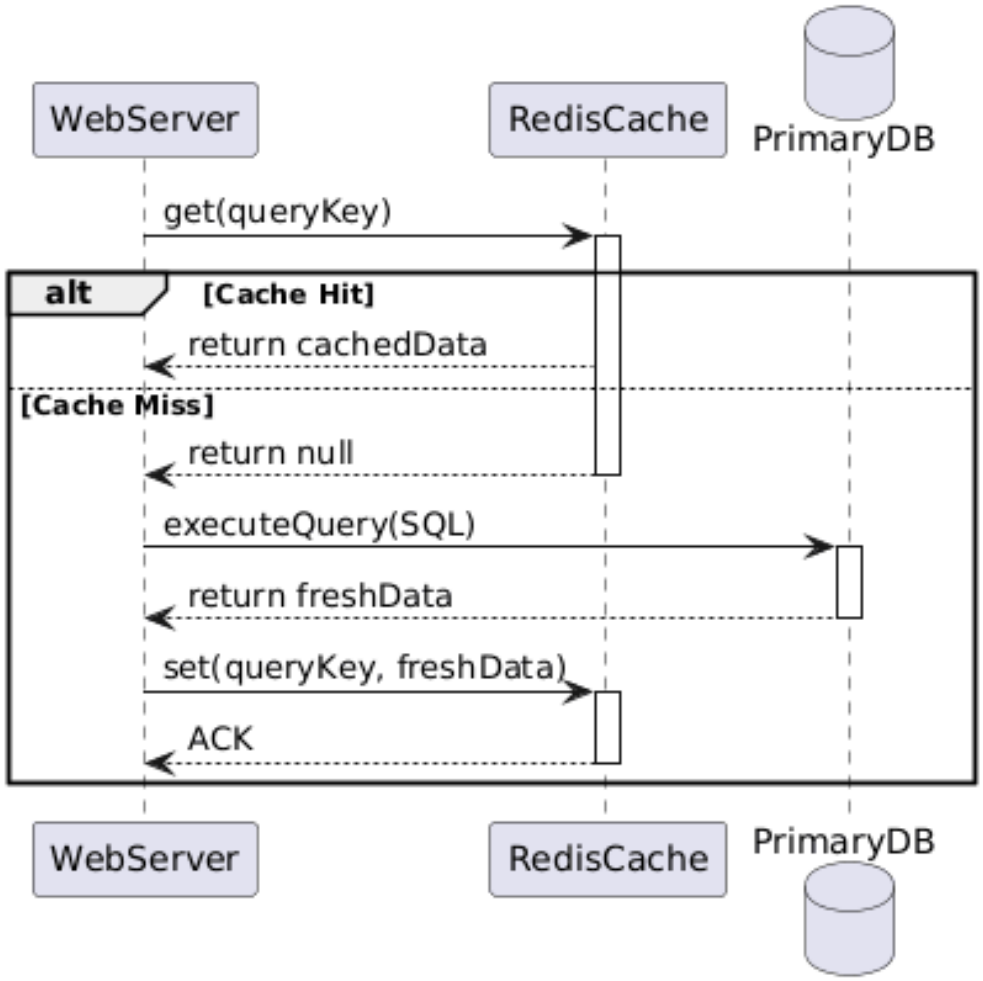

Diagram 5

Which object initiates the first action in this specific interaction?

a) RedisCache
b) PrimaryDB
c) **WebServer**
d) The user

25. Review the diagram below.

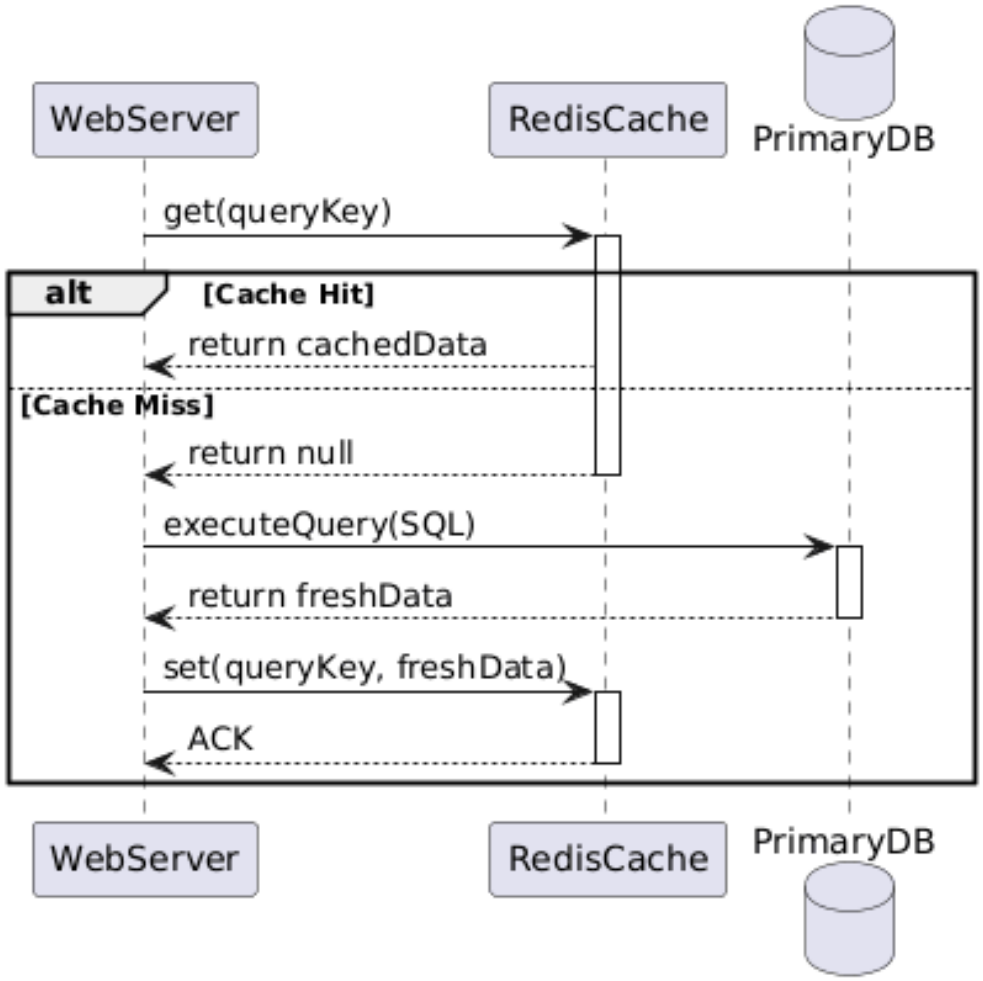

Diagram 5

What do the vertical dashed lines dropping down from the object names represent?

a) Data flow constraints
b) **The object's lifeline over time**
c) Method signatures
d) Network connections

26. Review the diagram below.

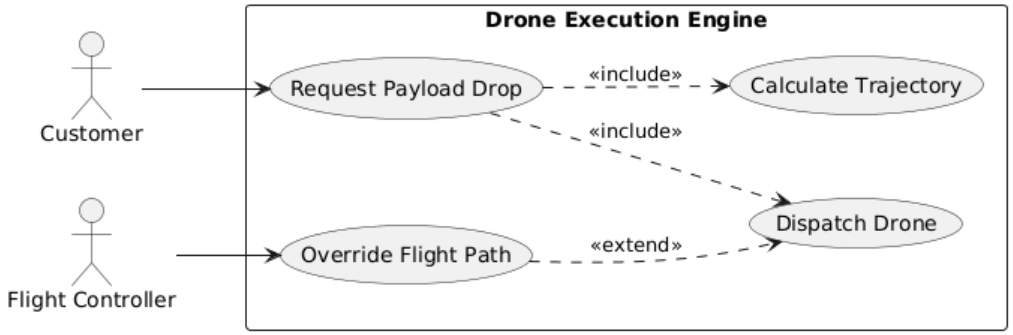

Diagram 6

What does an `«include»` relationship signify in a use case diagram?

a) The included use case is optional.

b) **The base use case always requires the execution of the included use case to complete its goal.**

c) The included use case is an alternative error path.

d) The included use case can only be triggered by a system administrator.

27. Review the diagram below.

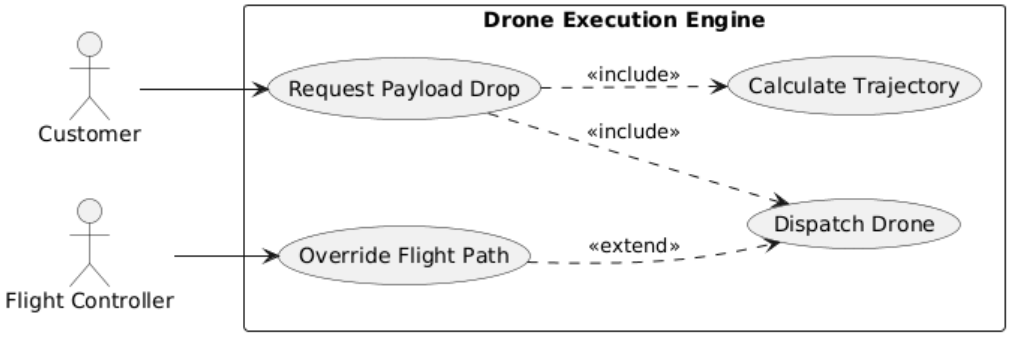

Diagram 6

How does the `«extend»` relationship differ from the `«include»` relationship?

a) It is used to show inheritance between actors.

b) It means the base use case must always run the extended use case.

c) **It represents optional or conditional behavior that can be inserted into the base use case.**

d) It connects a use case to an external database.

28. Review the diagram below.

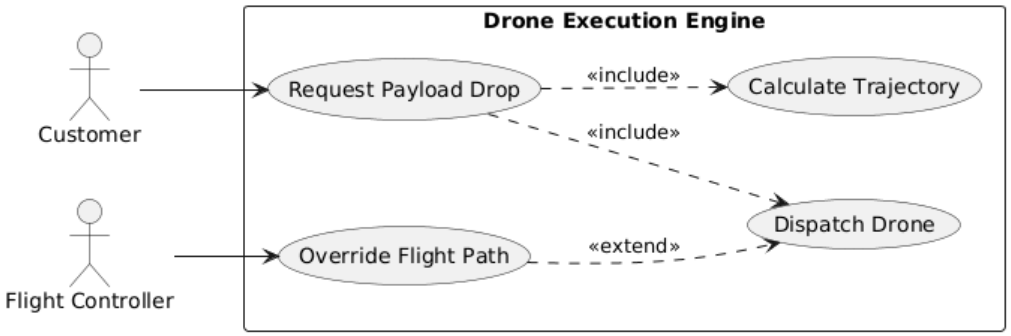

Diagram 6

What role do the "Customer" and "Flight Controller" figures play in this diagram?

a) They represent internal software modules.

b) They represent databases.

c) **They represent external actors that interact with the system.**

d) They represent use cases.

29. Review the diagram below.

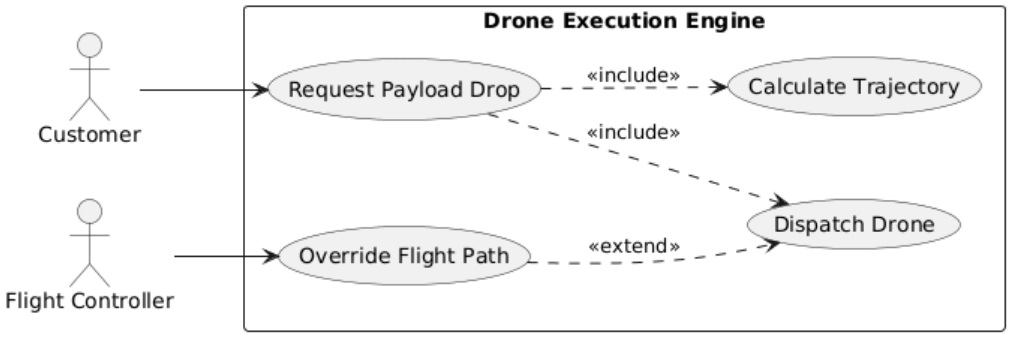

Diagram 6

Based on the lines connecting actors to use cases, who is allowed to trigger the "Override Flight Path" feature?

a) The Customer

b) **The Flight Controller**

c) Both the Customer and the Flight Controller

d) The Drone Execution Engine

30. Review the diagram below.

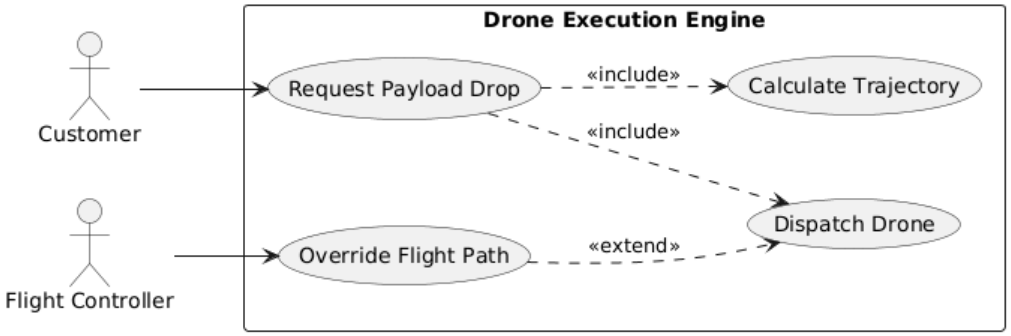

Diagram 6

What is the purpose of the large rectangle labeled "Drone Execution Engine"?

a) It represents a hardware server.

b) It shows the timeline of events.

c) **It defines the system boundary, separating internal functionality from external actors.**

d) It is a class containing methods and attributes.

31. Review the diagram below.

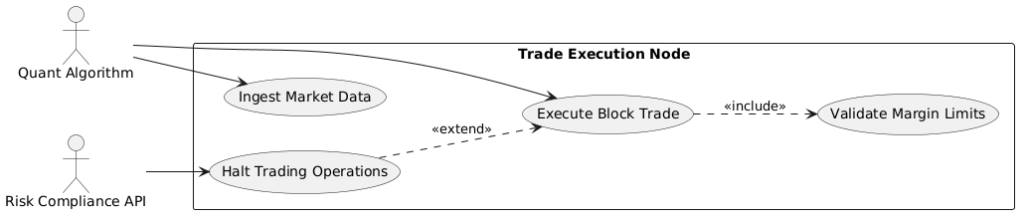

Diagram 7

Which use cases can the "Quant Algorithm" actor directly initiate?

a) **Ingest Market Data and Execute Block Trade**
b) Execute Block Trade and Validate Margin Limits
c) Halt Trading Operations
d) All use cases in the system

32. Review the diagram below.

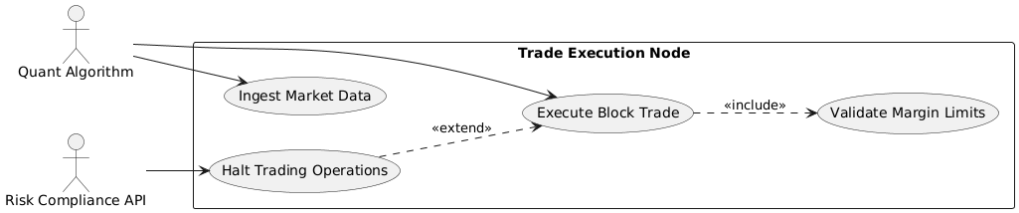

Diagram 7

Because "Execute Block Trade" has an `«include»` relationship with "Validate Margin Limits", what is true about the execution flow?

a) Validating limits happens independently of trades.
b) **Every time a block trade is executed, the margin limits must also be validated.**
c) Margin limits are only validated if an error occurs.
d) Validating margin limits is an optional step chosen by the actor.

33. Review the diagram below.

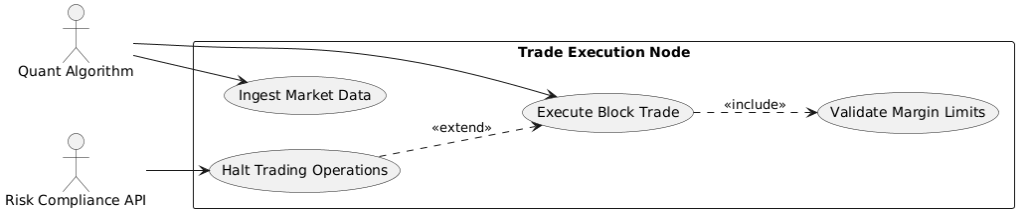

Diagram 7

If the "Risk Compliance API" decides to trigger "Halt Trading Operations", how does this affect "Execute Block Trade"?

a) It deletes the trade from the database.

b) It speeds up the trade processing.

c) **It acts as an extension, meaning the halt operation can conditionally interrupt or modify the standard trade execution.**

d) It converts the trade into market data.

34. Review the diagram below.

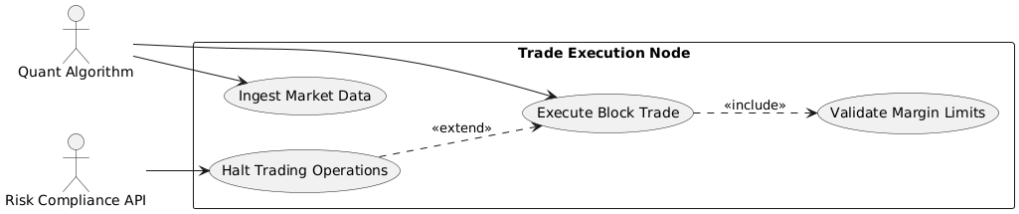

Diagram 7

Can the "Risk Compliance API" directly trigger the "Ingest Market Data" use case?

a) Yes, because it is an actor.

b) **No, there is no association line connecting them.**

c) Yes, through the extend relationship.

d) Only if the Quant Algorithm fails.

35. Review the diagram below.

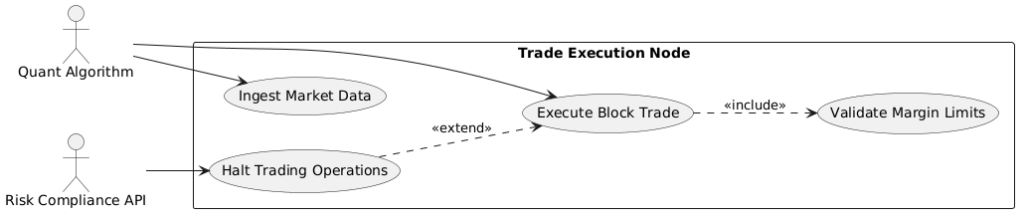

Diagram 7

In object-oriented analysis, what is the primary goal of creating a use case diagram?

a) To write the actual source code for the software.

b) To plan the database schema and tables.

c) **To capture the functional requirements of the system from the user's perspective.**

d) To define the exact order of method calls.

Answer Key

1. b

2. b

3. b

4. c

5. a

6. b

7. b

8. b

9. c

10. d

11. c

12. c

13. b

14. a

15. c

16. a

17. b

18. c

19. c

20. b

21. c

22. c

23. a

24. c

25. b

26. b

27. c

28. c

29. b

30. c

31. a

32. b

33. c

34. b

35. c